EE5203 L12 Study Sheet

System integration and the project clinic - Basri Erdogan

Essential question: Every part works alone and the system does not. What kind of failure is that, and what method finds it — given that you cannot stop the program to look?

What you should be able to do

- Build a resource map and spot a conflict in it before it reaches the bench.
- Compute a timing budget and judge whether a design fits.
- Distinguish component, resource, and timing failures by symptom.
- Instrument a running system with telemetry, a GPIO pin, and counters.
- Bisect a fault in code and in history.
- Write a requirement that can actually be tested.

Three species of failure

Species	Symptom	Found by
Component	one thing wrong, always, from the first run	ordinary debugging
Resource conflict	two subsystems fight over one resource	the resource map
Timing	works most of the time; fails under load	the timing budget

The species tells you which tool to reach for. Reaching for a breakpoint on a timing failure is how afternoons disappear.

The resource map

One row per resource, exactly one owner each.

Resource	Pin(s)	Owner
TIM2 CH1	PA5	LED PWM
TIM3 CH1	PA6	servo PWM
TIM4 CH1/2	PB6	input capture
ADC1 IN0/IN1	PA0, PA1	knob, light sensor
USART2	PA2, PA3	telemetry + commands
I2C1	PB8, PB9	display
SysTick	—	HAL, `HAL_GetTick`

The four conflicts that actually happen:

1. **One pin, two functions.** PA5 is LD2, TIM2_CH1, *and* SPI1_SCK. Changing your mind in CubeMX can leave the old init behind.
2. **One timer, two frequencies.** Channels share `PSC` and `ARR`. A 1 kHz LED and a 50 Hz servo cannot share a timer.
3. **`HAL_Delay` reachable from an interrupt.** It waits on SysTick; at equal or higher priority it never returns.
4. **Two ADC channels assumed simultaneous.** One converter, one multiplexer — the samples are separated by at least a conversion time.

The timing budget

$$\text{load} = \frac{\text{cost per occurrence} \times \text{rate}}{1 \text{ second}}$$

Activity	Cost	Rate	Load
Tick interrupt	~2 µs	1000 Hz	0.2 %
ADC read	~20 µs	100 Hz	0.2 %
UART line, 18 ch @ 115 200	1.56 ms	10 Hz	1.6 %
LCD line, 16 ch @ 100 kHz	7.2 ms	2 Hz	1.4 %

Useful reference figures from earlier lectures:

- UART: chars × 10/baud seconds per line.
- I2C LCD: 45 bits per character (4 data bytes + address) → 450 µs at 100 kHz, 112 µs at 400 kHz.
- Above ~50 % total, you need a plan. Above 100 %, it is a design error.

Measure rather than guess: toggle a spare GPIO around a section and capture it — on a scope, or on the input-capture channel you built in L08.

Interrupt discipline, in one place

1. A callback **records**; `main()` **decides**.
2. No `HAL_Delay`, no blocking transmit, no loop of unknown length.
3. Shared variables are `volatile`.
4. A callback longer than its own period is a design that cannot work.

Nearly every "random" integration failure traces to one of these four.

Debugging what you cannot stop

Instrument	Cost	Shows
Telemetry (L10)	~1.6 ms/line	state over time, plottable
GPIO toggle	nanoseconds	exact timing of a section
Event counters	negligible	overruns, NACKs, missed deadlines

Print counters once a second. "It hangs sometimes" is not a bug report; "it hangs after 400–600 s and `ore_count` is non-zero when it does" is one.

Bisection

When you do not know why, **halve it**:

1. Disable half the subsystems. Does the fault survive?
2. Yes → it is in the remaining half. No → in what you removed, or the interaction between the two.
3. Repeat. Six halvings isolate one item out of sixty.

The same method applies to **time**: `git bisect` between the last known-good commit and now. Guessing scales linearly with program size; bisection scales logarithmically.

The physical layer

- **Common ground** between every supply. Missing it fails silently.
- **Current budget.** A stalling servo draws an amp; the ST-LINK's 3.3 V rail supplies a few hundred milliamps.
- **Brownout.** The board resets and comes back clean, so the log shows a fresh start and no error at all.
- **Loose jumpers.** Intermittent and position-dependent — indistinguishable from a software race until you wiggle them.

Testable requirements

Not testable	Testable
"The display is fast"	"The display updates within 500 ms of a knob change"
"It reads temperature"	"Within ±2 °C of a reference between 10 and 40 °C"
"It is reliable"	"Runs 30 minutes with no reset and no lost telemetry lines"

If you cannot describe the measurement, you cannot claim the requirement — and the L13 defence will ask you to run it.

Common mistakes

- No resource map; the conflict is found at the demonstration.
- No timing budget; a refresh rate that cannot fit in a second.
- Blocking inside a callback.
- Changing three things at once, then not knowing which helped.
- No commits, so the version that worked is unrecoverable.
- Shared ground forgotten.
- Leaving integration to week 14.

Mastery self-check

- ☐ I can build a resource map and find a conflict in it.
- ☐ I can compute a load percentage from a cost and a rate.
- ☐ I can name the species of failure from a symptom.
- ☐ I can instrument a system three different ways.
- ☐ I can describe a bisection I actually performed.
- ☐ I can turn a vague claim into a testable requirement.

Five review questions

1. A team runs a 1 ms tick, a 20-character UART line at 20 Hz, and a 16-character LCD refresh at 5 Hz over 100 kHz I2C. Compute each load and the total. Which dominates?
2. The servo jitters only while the display updates. Name the two most likely species and the test that distinguishes them.
3. A system has six subsystems and one intermittent fault. How many bisection steps are needed in the worst case? Show the reasoning.
4. Rewrite as testable requirements: “the logging is reliable” and “the motor responds quickly.”
5. Telemetry runs perfectly for eight minutes and then stops for good, while the LED keeps breathing. Give the most likely cause, the counter you would add to confirm it, and the fix.